A receptionist should be able to register a patient in seconds. A doctor should be able to call the next patient with one clear action. A clinic owner should be able to manage doctors and working hours without technical knowledge. A patient should be able to check their queue position from a phone without creating an account.

At the same time, the platform owner should be able to manage hundreds of independent clinics through one SaaS platform.

This blueprint defines the product vision, users, permissions, workflows, features, architecture, screens, business model and development priorities. It is intended to be the foundation for the new build.

Part 1 — Product definition

QueuePilot

Simple queues. Better clinic experiences.

A multi-tenant, cloud-based clinic queue, appointment and patient-flow management SaaS.

The product in one sentence:

QueuePilot helps clinics register patients, organise doctor queues, manage appointments, guide patients through their visits and understand daily clinic activity — without complicated software.

What kind of product is it?

QueuePilot is a Software as a Service (SaaS) platform.

That means:

- One central QueuePilot application serves many independent clinics.
- Each clinic has its own account, users, doctors, patients, queues and settings.
- Clinics access the system through a web browser.
- Clinics subscribe to a plan to use the platform.
- QueuePilot manages the application, infrastructure, updates and platform-wide services.
- A clinic does not need to install or maintain its own server.
- Features and usage limits can vary according to the clinic's subscription.
- The platform owner can manage all tenants from a separate administration environment.

A small clinic with one doctor should be able to use QueuePilot without configuring a complicated system. A large clinic with several doctors, departments and branches should be able to grow into more advanced capabilities.

The product's central concept

A clinic has doctors. Each doctor working on a particular day has a queue. A patient receives a token belonging to that queue.

For example:

Doctor	Queue prefix	Example token
Dr. Nuwan Perera	A	A001
Dr. Amara Silva	B	B001
Dr. Imran Farook	C	C001

A token represents a patient's place in one doctor's queue for that day. The system manages its progress from registration to completion.

The existing guide defines nine internal token states and enforces valid transitions in the domain layer. We'll preserve the important business rules while presenting users with simpler language and fewer choices.

Part 2 — The people who use QueuePilot

The new system must be designed around five main user experiences. They should not all see the same dashboard.

1. Receptionist

Manages patient arrivals, registration, appointments and daily queues.

2. Doctor

Calls patients, manages consultations and controls their own queue.

3. Clinic owner / manager

Manages the clinic, staff, doctors, schedules, subscriptions and reports.

4. Patient

Books appointments, receives tokens and tracks their own queue position.

5. QueuePilot platform owner / administrator

Manages clinics, subscriptions, platform features, payments and support.

1. Receptionist experience

The receptionist is one of the most important users. The software must make their daily work faster, not give them additional administrative work.

Main responsibilities:

- See all doctors working today.
- See which doctors are consulting, available, paused or not started.
- Register walk-in patients.
- Find existing patients.
- Issue queue tokens.
- Check in patients with appointments.
- Book appointments on behalf of patients.
- View and manage appointments.
- See each doctor's waiting list.
- Call or recall patients where permitted.
- Skip or mark patients as no-show.
- Cancel or transfer a visit when authorised.
- View patient details and previous visits.
- Print token slips or share tracking links.

Receptionist's ideal daily workflow:

- Open QueuePilot.
- See today's clinic overview.
- Patient arrives.
- Search for the patient or add a new one.
- Select the doctor.
- Press Issue token.
- Give the patient their token.
- Continue serving the next patient.

The receptionist should not have to open several pages to perform this workflow.

Receptionist navigation:

- Overview
- Queue

- Appointments
- Patients

Other functions should appear only if the receptionist has permission or needs them.

Restrictions:

- Cannot manage subscriptions or billing.
- Cannot invite staff or change clinic-wide settings.
- Cannot manage other clinics.
- Cannot assign clinical priority by default. Clinical priority should be a clinician-authorised decision with a recorded reason.
- Cannot bypass queue rules simply by changing the screen.

2. Doctor experience

The doctor interface must be even simpler than the receptionist interface.

A doctor should not have to manage patient registration, clinic configuration, billing or reports to perform their normal work.

Main responsibilities:

- View today's queue.
- See how many patients are waiting.
- See the currently called patient.
- See the next patient.
- Call the next patient.
- Recall a patient.
- Start a consultation.
- Complete a consultation.
- Skip a patient or mark no-show.
- Put a patient on hold.
- Resume a patient from hold.
- Transfer a patient to another doctor when permitted.
- Assign clinical priority with a reason.

- Pause and resume their own session.
- Review their own queue and visit history.

Doctor's ideal daily workflow:

- Open the doctor workspace.
- See the queue and session status.
- Press Start session when ready.
- Press Call next.
- Patient arrives.
- Press Start consultation.
- Complete the consultation.
- Press Complete or Complete & call next.

The doctor must remain in control of when the next patient is called. Completing a consultation must not automatically call another patient unless the doctor explicitly chooses that action.

Important session rule:

Receptionists can issue tokens before a doctor starts their session. However, QueuePilot must not allow the doctor to call patients until the session has started.

Doctor navigation:

- Today's queue
- My schedule
- My patients / visit history, where permitted
- My profile

No clinic-wide configuration or platform administration should be visible.

3. Clinic owner / manager experience

The clinic owner controls the clinic's configuration and daily operation.

The owner should not need a technical background to manage the platform.

Main responsibilities:

- Manage clinic details.

- Add, edit and deactivate doctors.
- Assign doctors to rooms and departments.
- Set doctor schedules.
- Manage clinic opening hours and holidays.
- Configure queue prefixes.
- Invite receptionists and other authorised staff.
- Manage user access.
- Review clinic reports.
- View audit activity.
- Manage notification preferences.
- Configure appointment policies.
- Manage subscription and invoices.
- Manage branches if the clinic operates multiple locations.
- Access advanced queue settings when necessary.

Owner navigation:

- Overview
- Appointments
- Patients
- Doctors & rooms
- Reports
- Team
- Clinic settings
- Subscription

Advanced configuration should be placed inside settings, not mixed into the daily dashboard.

4. Patient experience

Patients should not have to understand the internal QueuePilot system.

Most patient interactions should work without an account.

Patient capabilities:

- View a clinic's public booking page.
- Select a doctor.
- View genuinely available appointment slots.
- Request an appointment.
- Receive booking confirmation.
- Manage an appointment using a secure link.
- Check in at the clinic.
- Receive a queue token.
- Scan a QR code on the token slip.
- Track their own queue position.
- View an estimated waiting-time range when sufficient data is available.
- See when it is their turn.
- Cancel or reschedule where permitted.
- Rate their visit afterwards, if the clinic enables feedback.

Patient tracking must show:

- Their token number.
- Doctor and room.
- Current status.
- Patients ahead of them.
- Estimated waiting range, if available.
- Last updated time.
- Simple instructions.

Patient privacy:

Patients must never see other patients' names, phone numbers, personal details or private visit history.

Public tracking links must be unpredictable, purpose-limited, revocable where supported and time-limited. Your existing guide specifies an 18-hour tracking-link expiry; the new implementation should retain that protection unless deliberately changed after a security review.

5. QueuePilot platform owner

The platform owner manages the SaaS business itself, not the day-to-day operations of individual clinics.

Responsibilities:

- Register and onboard clinics.
- View all registered clinics.
- Activate, suspend and reinstate clinic accounts.
- Manage clinic organisations and branches.
- Create and edit subscription plans.
- Manage feature entitlements.
- Review payment submissions.
- Manage invoices and billing status.
- View platform-wide usage.
- Monitor application health.
- Monitor notification workers and background jobs.
- Manage support requests.
- Review incidents.
- Manage platform settings.

The platform owner has a separate dashboard and a separate set of permissions.

Platform administration must not become an extension of the receptionist dashboard.

Part 3 — The complete feature set

The following is the target product scope. Some capabilities already exist in your previous implementation, while others are design and implementation recommendations for the rebuild. We should verify each feature against the actual source code before deciding what can be reused.

A. Daily clinic operations

Feature	What it does
Live clinic overview	Shows today's clinic activity
Doctor queue overview	Shows every doctor's current queue
Walk-in registration	Registers patients arriving without appointments

Token generation	Issues unique queue tokens
Queue management	Manages waiting patients
Patient check-in	Converts an eligible appointment into a queue visit
Queue pause / resume	Temporarily stops or resumes calling
Session management	Starts, pauses and closes doctor sessions
Visit details	Shows one patient's queue journey
Queue history	Displays completed and other previous visits

B. Patient management

- Patient registration.
- Existing patient search.
- Duplicate prevention for active visits.
- Patient profile.
- Patient contact details.
- Patient ID.
- Visit history.
- Patient record updates.
- Patient-specific tracking link.

QueuePilot is not intended to become a full electronic medical record (EMR). Do not add prescriptions, complex clinical notes, diagnoses or hospital-level medical records unless the product scope changes deliberately.

C. Queue engine

The internal engine must continue supporting:

- Waiting.
- Called.
- In consultation.
- Completed.
- Skipped.
- No-show.
- On hold.

- Transferred.
- Cancelled.

Users should see friendly labels such as:

Internal state	User-facing label
waiting	Waiting
called	Called — please proceed
in_consultation	With doctor
completed	Completed
skipped	Skipped
no_show	Not present
on_hold	On hold
transferred	Moved to another queue
cancelled	Cancelled

The UI should display only actions that are valid for the current state and the user's role.

D. Appointment management

- Public clinic booking page.
- Receptionist-created appointments.
- Doctor selection.
- Available appointment slots.
- Schedule and holiday checks.
- Appointment confirmation.
- Appointment check-in.
- Appointment rescheduling.
- Appointment cancellation.
- Booking management links.
- Appointment reminders when enabled.

An appointment and a queue token are different records. An appointment reserves a time; checking in creates or associates the actual queue visit according to the clinic's policy.

E. Doctors, rooms and departments

- Doctor profiles.
- Doctor availability.
- Doctor schedules.
- Doctor leave.
- Consultation rooms.
- Optional departments.
- Doctor-to-room assignment.
- Doctor-to-department assignment.
- Daily queue creation.
- Queue prefixes.

A small clinic should be able to operate with one doctor and no department setup.

F. Waiting room and public displays

- Main waiting-room TV display.
- Department-specific display.
- Public queue board.
- Patient position lookup.
- Kiosk check-in.
- Token slip with QR code.
- Automatic display refresh.

These screens must never expose private patient details.

G. Notifications

- Appointment confirmation.
- Appointment reminder.
- Token confirmation.
- Patient turn notification.
- Cancellation notification.

- SMS integration.
- Email integration.
- WhatsApp integration where configured.
- Notification history.
- Delivery failure tracking.
- Message templates.
- Usage tracking.

Notification failures must not stop a patient from being registered or a doctor from completing a consultation.

H. Clinic management

- Clinic profile.
- Working hours.
- Holidays.
- Staff.
- Roles and permissions.
- Doctors.
- Rooms.
- Departments.
- Queue preferences.
- Appointment preferences.
- Reports.
- Audit activity.
- Billing and subscription.
- Branch management.

I. Advanced capabilities

Keep these as optional capabilities rather than mandatory steps for every clinic:

- Clinical priority rules.
- Priority reason and audit.

- Patient transfers.
- Hold and resume.
- Custom workflows.
- Offline token reservation.
- Offline reconciliation.
- Advanced reports.
- Integrations.
- Organisation management.
- Multiple branches.
- Advanced notification configuration.
- Incident management.

J. Platform administration

- Clinic onboarding.
- Tenant management.
- Organisations.
- Plans.
- Billing.
- Payment verification.
- Feature entitlements.
- Support requests.
- Incident management.
- Platform health.
- Usage monitoring.
- Platform audit.

Part 4 — How the SaaS platform should work

QueuePilot must be designed as a multi-tenant SaaS application, not as separate installations for every clinic.

Tenant structure

QueuePilot platform

One application · One platform administration environment

Clinic A

Tenant A

Doctors

Patients

Queues

Clinic B

Tenant B

Doctors

Patients

Queues

Clinic C

Tenant C

Doctors

Patients

Queues

Separate tenant data and permissions, shared application and infrastructure.

Each clinic must have its own:

- Clinic identity and profile.
- Users and roles.
- Doctors and rooms.
- Patients and patient history.
- Appointments.
- Queue records.
- Settings.

- Reports.
- Subscription.
- Notification configuration.
- Audit records.

Tenant isolation is mandatory. A receptionist from Clinic A must never be able to access Clinic B's patients by changing an ID in a URL or API request.

Organisations and branches

Support two levels:

- An independent clinic operating on its own.
- An organisation containing multiple clinics or branches.

A clinic with one location must not be forced to create an organisation.

Subscription model

Use configurable subscription plans, not hardcoded plan restrictions throughout the code.

Your previous guide included these example plans:

Plan	Example monthly price
Starter	LKR 3,500
Professional	LKR 7,900
Clinic Plus	LKR 14,900
Enterprise	Contact sales

The final prices, limits and feature entitlements must be editable from the platform administration panel.

Possible plan-controlled capabilities:

- Number of doctors.
- Number of staff.
- Number of branches.
- Appointment booking.
- Patient tracking.
- SMS usage.

- Public displays.
- Advanced reports.
- Offline desk.
- Organisation features.

A feature entitlement must be enforced by the backend. Hiding a button is not sufficient.

Part 5 — The complete screen map

These are the screens and experiences the new system should support. Some should be separate pages; others should be tabs, drawers, dialogs or alternate states of the same page.

Area	Screens
Authentication	Login, forgot password, reset password, email verification, profile
Onboarding	Welcome, clinic details, add first doctor, working hours, invite receptionist
Reception	Overview, registration, queue board, visit details
Patient management	Patient directory, patient profile, visit history
Appointments	Appointment list, booking form, appointment details
Doctor workspace	Session not started, session active, patient called, consultation active, consultation completed
Clinic setup	Doctors, rooms, departments, schedules, breaks, holidays
Clinic administration	Staff, access, clinic profile, settings
Patient experiences	Public booking, manage booking, tracking, kiosk
Public displays	Waiting room, department board, public queue board
Notifications	Notification centre, templates, integrations
Reports	Daily reports, doctor reports, waiting-time reports, activity reports
Advanced operations	Audit, incidents, offline desk, data export, support
Organisation	Organisation overview, branches, branch details
Subscription	Current plan, invoices, payment details
Platform administration	Overview, clinics, organisations, plans, features, payments, support, health

Your first 10 approved screen designs should remain the starting visual reference. The rest of the screens should be designed in the same language, not as a different product.

Part 6 — The technology and architecture

Given your existing project and development approach, I recommend rebuilding the user experience without unnecessarily changing the technology stack.

Recommended stack

Layer	Technology
Backend	Native PHP
Database	MySQL
Frontend	HTML, CSS, JavaScript
Styling	Tailwind CSS or your established CSS approach
Icons	Lucide
Server	Apache or Nginx
API	PHP JSON endpoints where needed
Background work	PHP queue worker / scheduled tasks
Notifications	SMS, email and optional WhatsApp gateways
Public interfaces	Responsive web pages and PWA capabilities

You do not need Laravel, React, microservices or multiple separate applications just to make QueuePilot professional. Your existing native PHP modular-monolith approach can support this product.

Suggested project structure

```
queuepilot/
?
??? app/
?   ??? Core/
?   ?   ??? Auth.php
?   ?   ??? Database.php
?   ?   ??? Router.php
?   ?   ??? Request.php
?   ?   ??? Response.php
?   ?   ??? Session.php
?   ?
?   ??? Modules/
?   ?   ??? Auth/
?   ?   ??? Clinics/
?   ?   ??? Users/
?   ?   ??? Patients/
?   ?   ??? Doctors/
?   ?   ??? Rooms/
?   ?   ??? Queues/
?   ?   ??? Appointments/
?   ?   ??? Sessions/
?   ?   ??? Notifications/
?   ?   ??? Reports/
?   ?   ??? Billing/
?   ?   ??? Organisations/
?   ?   ??? Support/
?   ?   ??? Platform/
```

```
?   ?
?   ??? Policies/
?   ??? Middleware/
?   ??? Services/
?   ??? Support/
?
??? config/
?
??? database/
?   ??? migrations/
?   ??? seeds/
?   ??? schema/
?
??? public/
?   ??? assets/
?   ??? uploads/
?   ??? index.php
?
??? resources/
?   ??? views/
?   ?   ??? layouts/
?   ?   ??? components/
?   ?   ??? receptionist/
?   ?   ??? doctor/
?   ?   ??? owner/
?   ?   ??? patient/
?   ?   ??? platform/
?   ?
?   ??? css/
?   ??? js/
?
??? routes/
??? storage/
?   ??? logs/
?   ??? cache/
?   ??? exports/
?
??? tests/
??? tools/
??? .env.example
??? README.md
```

This is a suggested target structure, not a requirement to move every existing file immediately. Organise code by feature when it helps maintainability.

Important backend modules

Authentication: login, logout, password recovery, verification, session security.

Tenant management: resolve the active clinic, enforce clinic boundaries and manage organisation relationships.

Patient management: create, search, update and retrieve patient records.

Queue service: issue tokens, transition states, order waiting lists, transfer and cancel visits.

Appointment service: check availability, create bookings, check in, reschedule and cancel.

Session service: doctor schedules, start and pause sessions, breaks and closures.

Notification service: create messages, deliver them asynchronously, retry failures and track usage.

Billing service: subscriptions, invoices, payment verification and entitlements.

Reporting service: aggregate queue, patient, doctor and waiting-time information.

Audit service: record sensitive and operational changes.

Part 7 — Database design principles

Design the database around real clinic operations rather than around the screens.

At minimum, consider these entities:

Entity	Purpose
organisations	Parent organisations
clinics	Individual clinic tenants
branches	Clinic locations
users	Login accounts
clinic_users	User-to-clinic memberships and roles
patients	Clinic-scoped patient register
doctors	Doctor profiles
departments	Optional clinic departments
rooms	Consultation rooms
doctor_schedules	Recurring working schedules
doctor_leaves	Doctor absence
clinic_holidays	Clinic closures
sessions	Doctor's daily working session
queues	Doctor's daily queue
queue_entries	Patient token and visit
queue_events	Visit activity history
appointments	Appointment bookings
appointment_slots	Available time slots, if separately modelled
priority_rules	Configurable priority behaviour

notification_outbox	Pending and retryable messages
notification_logs	Delivery history
subscriptions	Clinic subscription
plans	SaaS plans
feature_entitlements	Plan and clinic feature access
invoices	Billing records
payments	Submitted and verified payments
audit_logs	Important system activity
support_tickets	Clinic support requests
incidents	Operational problems
offline_desks	Offline token reservations and reconciliation
public_display_keys	Public display access
tracking_tokens	Patient tracking access

This is a logical starting point, not a final migration script. Several entities may already exist in your current database and should be reused or refined.

Database rules

- Every clinic-owned record must have a reliable tenant relationship.
- Use foreign keys where appropriate.
- Use unique constraints for identifiers that must not collide.
- Index common searches such as patient name, phone number, token and appointment date.
- Store dates and times consistently, respecting each clinic's timezone.
- Keep appointment records separate from queue visits.
- Preserve queue event history.
- Avoid deleting important operational records just to clean up a screen.
- Use transactions for operations that affect multiple records.

Part 8 — Essential business rules

These are not optional visual preferences. They are rules the new application must enforce.

Queue rules

- Each doctor working on a given day has a queue.

- Tokens belong to a particular queue and date.
- A token must not be duplicated within its queue.
- A token can be issued before the doctor starts the session.
- Calling patients is blocked until the session is started.
- Only valid transitions are allowed.
- A patient cannot be completed directly from waiting.
- A repeated or stale action must not call the same patient twice.
- Priority must have an authorised reason and audit record.
- Transferred visits must retain their history.
- A completed visit cannot be accidentally called again.
- Cancelling a doctor session requires clear confirmation and an appropriate reason.

Appointment rules

- Appointment availability must respect doctor schedules.
- Clinic working hours must be respected.
- Clinic holidays must be respected.
- Doctor leave must be respected.
- Existing bookings must be considered.
- The booking window and appointment policy must be enforced.
- A patient cannot book an unavailable slot.
- Appointment check-in must create or link a queue entry correctly.
- Cancellation and rescheduling must preserve the appropriate history.

Permission rules

Action	Owner	Receptionist	Doctor	Platform admin
Register walk-in patient	Yes	Yes	No	Administrative access only
Issue token	Yes	Yes	No	Administrative access only
Call / start / complete	Yes	Yes	Yes	Support access only
Set clinical priority	Yes	No	Yes	Only when explicitly authorised
Manage patients	Yes	Yes	Own patients	No routine clinical access

Manage appointments	Yes	Yes	Own appointments	No routine clinical access
View reports	Yes	Basic	Own reports	Platform reports
Manage doctors and rooms	Yes	No	No	Platform-level support only
Manage clinic staff	Yes	No	No	Platform-level administration
Manage clinic subscription	Yes	No	No	Yes
Manage all clinics and plans	No	No	No	Yes

The permission model should be enforced server-side. The table describes default permissions; any optional delegation must be explicitly configured.

Part 9 — The most important usability requirements

These requirements should influence every screen.

1. Make the common actions extremely fast

- Patient registration should be short.
- Existing patient search should be immediate.
- Issuing a token should require very few steps.
- Calling the next patient should be obvious.
- Completing a consultation should be one clear action.
- Checking in an appointment should not require re-entering patient details.

2. Use progressive disclosure

Do not show all nine queue states and all possible actions at once.

For example, when a patient is waiting, show the most useful action first: Call.

Put less common actions such as transfer, hold and cancellation in an overflow menu or detail panel.

3. Use sensible defaults

- Automatically select today's date where appropriate.
- Show currently working doctors first.
- Preselect the clinic's default settings.
- Use the existing patient record when found.
- Hide optional configuration until needed.

4. Give useful feedback

Every important action should produce a clear result:

- Patient registered.
- Token issued.
- Patient called.
- Consultation started.
- Visit completed.
- Appointment confirmed.
- Changes saved.
- Notification failed.

Do not leave users wondering whether a button worked.

5. Handle mistakes safely

- Confirm destructive actions.
- Make cancellation reasons clear where needed.
- Prevent duplicate submissions.
- Prevent accidental double-calling.
- Show helpful validation beside the relevant field.
- Do not silently discard unsaved changes.

6. Make the application comfortable

- Responsive desktop, tablet and mobile layouts.
- Keyboard-friendly registration.
- Touch-friendly doctor controls.
- Readable typography.
- Clear contrast.
- No excessive animations.
- No unnecessary dashboard charts.
- No technical jargon.

7. Design for unreliable connectivity

Clinics may have slow or interrupted internet connections.

- Show when data is being refreshed.
- Display when information is stale.
- Never pretend offline data is live.
- Avoid losing a registration when a network request fails.
- Make duplicate submissions safe.
- Keep an explicit offline workflow if enabled.
- Reconcile reserved offline tokens carefully.

Part 10 — Security and reliability

QueuePilot handles patient information and must be built responsibly.

Authentication and access

- Secure password hashing.
- Session protection.
- CSRF protection.
- Rate limiting.
- Login abuse protection.
- Email verification where required.
- Role and resource-level authorisation.
- Secure password reset.

Tenant security

- Resolve clinic membership from the authenticated account.
- Never trust a clinic ID supplied by the browser without authorisation.
- Apply tenant scoping to every query.
- Enforce access to the particular patient, appointment, queue and branch.
- Test cross-tenant access attempts.

Patient privacy

- Use unpredictable tracking tokens.
- Limit tracking links to their intended purpose.
- Expire and revoke links appropriately.
- Never expose other patients' identities on public boards.
- Keep patient exports restricted to authorised users.
- Avoid logging sensitive patient details unnecessarily.

Queue reliability

- Use database transactions.
- Protect concurrent actions from two receptionists or a receptionist and doctor acting at once.
- Reject stale transitions.
- Use idempotency or equivalent protection for duplicate requests.
- Keep an audit trail for important changes.

Notifications

- Queue messages independently from the patient registration or consultation transaction.
- Use retry logic and delivery logs.
- Avoid duplicate notifications.
- Never let an SMS gateway outage stop the queue from working.

Platform reliability

- Automated database backups.
- Tested restoration procedures.
- Application error logs.
- Health checks.
- Background worker monitoring.
- Database migration tracking.
- Production environment separation.
- Secure environment variables.
- Incident logging.

Part 11 — The development roadmap

Do not try to build all advanced features before the basic clinic workflow works perfectly.

Recommended implementation phases

- Phase 1 — Foundation

  SaaS tenancy, authentication, clinic onboarding, users, roles, database and basic clinic configuration.

  Must work first

- Phase 2 — Core queue

  Doctors, sessions, queues, tokens, patient registration, queue actions and visit history.

  Must work first

- Phase 3 — Everyday interfaces

  Reception dashboard, quick registration, doctor workspace and live queue management.

  Must work first

- Phase 4 — Appointments and patients

  Patient directory, profiles, booking, appointment check-in, rescheduling and cancellation.

- Phase 5 — Patient experience

  Tracking links, QR tokens, public queue boards, waiting-room displays and kiosk.

- Phase 6 — Clinic management

  Staff, permissions, schedules, rooms, departments, breaks, holidays and settings.

- Phase 7 — SaaS operations

  Plans, subscriptions, invoices, payment verification, feature entitlements and platform admin.

- Phase 8 — Advanced features

  Notifications, integrations, reports, audit, offline operation, incidents, support and organisations.

- Phase 9 — Production readiness

  Security testing, tenant-isolation testing, concurrency testing, backups, recovery, performance and usability testing.

Definition of a successful first release

A small clinic should be able to:

- Register its clinic.

- Add one doctor.
- Add one receptionist.
- Set basic working hours.
- Open the clinic for the day.
- Register walk-in patients.
- Issue tokens.
- Start the doctor's session.
- Call and complete patients.
- See the queue update.
- Allow patients to track their own token.
- Review a simple daily report.

If these tasks are not easy, adding more advanced functionality will not make the product better.

Part 12 — The master prompt for building the new QueuePilot

The following is the main project brief you can provide to an AI coding assistant or use as the source document for your new development project. It consolidates the product direction, roles, SaaS model, features, workflows, architecture and engineering requirements.

QUEUEPILOT — COMPLETE PRODUCT BUILD SPECIFICATION

1. PROJECT IDENTITY

Product name: QueuePilot

Product type: Multi-tenant SaaS clinic queue and appointment management platform.

Primary purpose: Help clinics manage patient registration, doctor queues, appointments, consultations and patient communication through an extremely simple and efficient application.

Product philosophy:

"Simple queues. Better clinic experiences."

QueuePilot must be designed for ordinary clinics, including small clinics with one doctor and one receptionist. It must also support larger clinics with multiple doctors, departments and branches.

This is not a hospital management system or a full electronic medical record platform.

The central objective is to make daily clinic operations faster, easier and more organised.

2. CORE PRODUCT PRINCIPLE

Do not recreate a complicated management interface with a new visual design.

Redesign the product experience around the actual daily tasks of clinic staff.

The system should be powerful internally but simple externally.

A receptionist should be able to register a patient and issue a token within seconds.

A doctor should be able to see the next patient, call them and complete a consultation without navigating through unnecessary pages.

A clinic owner should be able to manage the clinic without technical knowledge.

A patient should be able to track their queue without registering an account.

A platform administrator should be able to manage all clinics and subscriptions through a separate platform administration interface.

Keep advanced capabilities available, but do not make them part of the everyday workflow.

3. SAAS BUSINESS MODEL

QueuePilot is a cloud-based multi-tenant SaaS platform.

One central application serves multiple independent clinic tenants.

Each clinic has separate:

- Clinic information
- Users and memberships
- Doctors
- Rooms
- Patients
- Appointments
- Queues
- Visits
- Settings
- Reports
- Audit records

- Subscription information

Clinics must never access each other's private data.

Support independent clinics and organisations containing multiple branches.

A clinic must not be forced to configure branches or organisations when it operates at one location.

Support configurable subscription plans and feature entitlements.

Example plans:

- Starter
- Professional
- Clinic Plus
- Enterprise

Example monthly prices from the previous product plan:

- Starter: LKR 3,500
- Professional: LKR 7,900
- Clinic Plus: LKR 14,900
- Enterprise: Contact sales

All prices, limits and entitlements must be configurable by the platform administrator.

Do not hardcode plan restrictions throughout the application.

Feature entitlements must be checked in backend services and not merely hidden in the frontend.

4. USER ROLES

Implement separate role-based experiences.

A. Receptionist

The receptionist manages daily patient flow.

Capabilities:

- View clinic overview.
- View doctors working today.
- View each doctor's queue.
- Search existing patients.
- Register new walk-in patients.
- Issue tokens.
- Check in appointments.

- Book appointments.
- View patient profiles.
- View visit history.
- Manage appointments.
- View queue status.
- Call or recall patients where authorised.
- Skip patients.
- Mark no-show.
- Cancel visits where permitted.
- Transfer visits where permitted.

Restrictions:

- Cannot manage clinic subscriptions.
- Cannot manage clinic-wide settings.
- Cannot invite or manage clinic staff.
- Cannot access platform administration.
- Cannot assign clinical priority by default.

B. Doctor

The doctor manages their own queue and consultations.

Capabilities:

- View today's queue.
- Start their session.
- Call the next patient.
- Recall a patient.
- Start consultation.
- Complete consultation.
- Complete and call next.
- Skip a patient.
- Mark no-show.
- Hold and resume patients.
- Transfer a patient when authorised.
- Assign clinical priority with a reason.
- Pause and resume their session.
- View their own queue history.
- View their own patients and appointments where permitted.

Restrictions:

- Cannot issue tokens.
- Cannot manage clinic-wide settings.
- Cannot manage staff or subscriptions.
- Cannot manage other doctors' queues without explicit permission.

The doctor can manage and cancel their own day's session according to the clinic's rules.

C. Clinic owner / manager

Capabilities:

- Manage clinic profile.
- Manage doctors.
- Manage rooms and departments.
- Manage staff.
- Manage permissions.
- Configure working hours.
- Configure schedules.
- Manage holidays and breaks.
- Configure queue settings.
- Configure appointment policies.
- View clinic reports.
- View audit history.
- Manage notifications.
- Manage branches.
- Manage subscriptions and invoices.

D. Patient

Patient capabilities:

- Use public appointment booking.
- Select doctors and available times.
- Request appointments.
- Receive booking confirmation.
- Manage bookings through secure links.
- Check in.
- Receive queue tokens.
- Scan QR codes.
- Track their own queue position.

- View their estimated waiting range where available.
- Cancel or reschedule where permitted.
- Submit visit feedback where enabled.

Patients should not need an account for basic booking or tracking.

E. Platform owner / administrator

Capabilities:

- Manage all clinic tenants.
- Activate, suspend and reinstate clinics.
- Manage organisations and branches.
- Manage plans and subscription prices.
- Configure feature entitlements.
- Review payments.
- Manage invoices.
- Monitor platform usage.
- Monitor application health.
- Monitor background jobs.
- Manage support tickets.
- Manage incidents.
- Review platform activity.

Use a separate platform administration interface.

5. CORE CLINIC WORKFLOW

The normal daily workflow must be:

1. Receptionist opens QueuePilot.
2. Receptionist sees today's clinic overview.
3. Patient arrives.
4. Receptionist searches for the patient or creates a new patient.
5. Receptionist selects a doctor.
6. Receptionist issues a token.
7. Patient receives a printed slip or tracking link.
8. Doctor starts their session.
9. Doctor calls the next patient.
10. Patient enters consultation.
11. Doctor starts consultation.

12. Doctor completes consultation.
13. Doctor chooses whether to call the next patient.
14. The queue updates for reception and public displays.

Tokens may be issued before a doctor starts their session.

Calling must remain blocked until the doctor starts the session.

Never automatically call another patient after completion unless the doctor explicitly selects "Complete & call next".

6. QUEUE ENGINE

Every doctor working on a particular day has a queue.

Each queue has:

- Clinic
- Doctor
- Date
- Token prefix
- Queue status
- Session relationship
- Queue entries

Tokens are unique within the appropriate queue and date.

Internal visit states:

- waiting
- called
- in_consultation
- completed
- skipped
- no_show
- on_hold
- transferred
- cancelled

Display friendly labels to users.

Only allow valid state transitions.

The backend domain layer must enforce transitions.

Do not rely on hidden or disabled frontend buttons for security.

Protect concurrent actions with database transactions and appropriate locking or equivalent concurrency control.

Reject stale and repeated operations safely.

Keep an event history for important visit actions.

Support priority, transfer, hold, resume, re-queue, skip, no-show and cancellation according to role permissions and state rules.

7. PATIENTMANAGEMENT

Implement:

- Patient registration.
- Patient search.
- Duplicate prevention.
- Patient ID.
- Patient profile.
- Contact details.
- Visit history.
- Patient updates.
- Queue registration.

Patient information must be clinic-scoped.

Do not build a full EMR.

8. APPOINTMENTS

Implement:

- Public clinic booking.
- Receptionist booking.
- Doctor selection.
- Real availability.
- Working-hour validation.
- Holiday validation.
- Doctor leave validation.
- Existing booking validation.
- Appointment confirmation.

- Appointment cancellation.
- Rescheduling.
- Secure booking management links.
- Appointment check-in.

An appointment and queue token are separate concepts.

Checking in should create or associate the correct queue visit while respecting the clinic's appointment priority policy.

9. DOCTORS, ROOMS AND SCHEDULES

Implement:

- Doctor profiles.
- Specialties.
- Rooms.
- Optional departments.
- Doctor-to-room assignments.
- Doctor-to-department assignments.
- Recurring schedules.
- Doctor leave.
- Clinic holidays.
- Daily sessions.
- Breaks.
- Queue prefixes.

Make the default setup easy for a clinic with one doctor.

10. PATIENT-FACING FEATURES

Implement:

- Public booking page.
- Booking management page.
- Patient tracking page.
- QR tracking links.
- Public waiting room board.
- Department display.
- Patient kiosk.

Patient tracking must show only the patient's own token and queue position.

Do not reveal other patients' names or contact details.

Tracking links must be unpredictable, purpose-limited and expire according to security configuration. The existing design uses an 18-hour expiry.

Waiting room displays must use large, readable text and must not expose private information.

11. NOTIFICATIONS

Support configurable integrations:

- SMS
- Email
- WhatsApp where available

Possible events:

- Appointment confirmation.
- Appointment reminder.
- Token issued.
- Patient turn approaching.
- Appointment cancelled.

Implement:

- Message templates.
- Notification history.
- Delivery status.
- Retry handling.
- Usage metering.
- Duplicate protection.

Notifications must be independent of queue progression.

A failed SMS must never prevent a patient from being registered or a consultation from being completed.

12. CLINIC MANAGEMENT

Implement:

- Clinic profile.
- Opening hours.
- Staff management.
- Doctor management.
- Rooms.

- Departments.
- Schedules.
- Breaks.
- Holidays.
- Queue preferences.
- Appointment policies.
- Reports.
- Audit history.
- Subscription management.
- Branch management.

Keep advanced configuration out of the normal receptionist and doctor workspace.

13. REPORTING

Provide understandable reports:

- Daily patients.
- Completed visits.
- Waiting patients.
- No-shows.
- Skipped visits.
- Average waiting time.
- Longest waiting time.
- Patients by doctor.
- Patients by hour.
- Doctor activity.
- Department activity where applicable.
- Satisfaction feedback where enabled.

Do not overwhelm small clinics with unnecessary analytics.

Waiting-time estimates must be presented as estimates, not guaranteed arrival times.

14. ADVANCED FEATURES

Support optional advanced capabilities:

- Clinical priority rules.
- Reason and audit for priority.
- Visit transfers.

- Hold and resume.
- Custom workflows.
- Offline token reservation.
- Offline reconciliation.
- Advanced reports.
- Organisation management.
- Multiple branches.
- Integrations.
- Incident management.
- Support requests.
- Data exports.

Do not force small clinics to configure these features.

15. PLATFORM ADMINISTRATION

Create a separate platform console.

Pages:

- Platform overview.
- Clinic management.
- Organisation management.
- Plans and billing.
- Feature entitlements.
- Payment verification.
- Support queue.
- Platform health.
- Incidents.
- Platform settings.

Show operational status only when backed by actual monitoring data.

Protect platform actions with strong permissions and audit logging.

16. USER INTERFACE PRINCIPLES

The product must feel:

- Simple.
- Fast.
- Calm.

- Modern.
- Professional.
- Comfortable.
- Easy to learn.

Use the approved QueuePilot designs as the visual reference.

Maintain a consistent design system:

- Light theme.
- Warm white backgrounds.
- Light grey surfaces.
- Navy or charcoal text.
- Restrained blue or teal primary actions.
- Green for successful states.
- Amber for attention.
- Red for destructive actions and errors.
- Clear typography.
- Consistent spacing.
- Subtle borders.
- Comfortable controls.
- Responsive layouts.

Do not make the interface look like a generic hospital management system.

Avoid unnecessary charts, oversized headings, decorative gradients, complex tables and excessive cards.

Use one primary action per screen.

Use plain language.

Show advanced actions only when relevant.

The interface must work comfortably on desktop, tablet and mobile.

Doctor controls must be especially touch-friendly.

17. ARCHITECTURE

Use a modular monolithic architecture.

Preferred technology:

- Native PHP.

- MySQL.
- HTML.
- CSS.
- JavaScript.
- Tailwind CSS or the established styling system.
- Lucide icons.

Do not introduce Laravel, microservices, React or a separate frontend/backend architecture without a clear technical requirement.

Separate:

- Core infrastructure.
- Authentication.
- Tenant management.
- Domain services.
- Controllers.
- Policies.
- Database migrations.
- Views.
- Frontend assets.
- Tests.

Use clear module boundaries.

Keep business rules in backend services and domain logic, not in view templates.

Use reusable UI components and role-specific layouts.

18. SECURITY

Implement:

- Secure authentication.
- Password hashing.
- CSRF protection.
- Rate limiting.
- Session security.
- Server-side authorisation.
- Tenant isolation.
- Resource-level access checks.
- Secure public tokens.

- Audit logging.
- Safe file uploads.
- Secure environment variables.
- Database backups.
- Error handling.

Every tenant-owned query must enforce clinic access.

Never trust a clinic ID or resource ID supplied by the browser without validating the authenticated user's permissions.

Use transactions and concurrency protection for queue operations.

19. RELIABILITY

Implement:

- Database transaction safety.
- Duplicate request protection.
- Stale action rejection.
- Background notification processing.
- Retry logic.
- Error logs.
- Health checks.
- Backup and restore procedures.
- Migration tracking.
- Monitoring.
- Clear offline and stale-data indicators.

Do not make cached data appear live.

20. DEVELOPMENT APPROACH

Build incrementally.

Phase 1:
Authentication, tenant isolation, clinic onboarding, users and roles.

Phase 2:
Doctors, sessions, queues, tokens and patient registration.

Phase 3:
Reception dashboard, quick registration, doctor workspace and queue management.

Phase 4:
Appointments and patient directory.

Phase 5:
Patient tracking, booking, kiosk and public displays.

Phase 6:
Clinic management, staff, schedules and settings.

Phase 7:
Subscriptions, billing and platform administration.

Phase 8:
Notifications, reports, audit, offline functionality, support and integrations.

Phase 9:
Security, concurrency, performance, usability and production testing.

Do not build all advanced features before the core clinic workflow is reliable and easy.

21. QUALITY ACCEPTANCE CRITERIA

The system is not ready merely because its pages load.

A small clinic must be able to:

- Create its account.
- Configure its clinic.
- Add a doctor.
- Invite a receptionist.
- Register patients.
- Issue tokens.
- Start the doctor's session.
- Call patients.
- Complete consultations.
- Manage appointments.
- Track queue status.
- Review daily activity.

A receptionist should understand the main interface after a short introduction.

A doctor should be able to manage a queue without extensive software training.

A patient should be able to track a token without logging in.

A clinic must never access another clinic's data.

Queue operations must remain correct when multiple authorised staff act at the same time.

The application must remain usable when notifications fail.

22. FINAL PRODUCT VISION

QueuePilot must not compete by having the most buttons or the most complicated configuration.

It should compete through simplicity, speed, reliability and ease of adoption.

Keep the powerful functionality in the background.

Make the common tasks effortless.

A clinic should be able to start small, operate comfortably and grow into more advanced capabilities only when it needs them.

Build QueuePilot as a professional, secure and scalable SaaS product, but never sacrifice everyday usability for technical complexity.

Part 13 — What I would change about the rebuild strategy

There is one important distinction I want to make.

You have already invested in the original QueuePilot. The previous guide describes a working application, not just a conceptual product. That means we should not assume everything needs to be rewritten.

My recommendation is:

Area	Rebuild approach
UI and navigation	Redesign around the approved screens
Reception workflow	Simplify significantly
Doctor workflow	Build a dedicated, minimal workspace
Patient experience	Make it mobile-first
Queue engine	Preserve and improve where necessary
Authentication	Reuse after security review
Tenant isolation	Review and strengthen
Database	Reuse compatible tables; migrate only where needed
Notifications	Keep independent from core queue actions
Billing and SaaS	Simplify the clinic-facing experience
Advanced features	Retain as optional capabilities

The first thing I would actually implement is not the billing system, reports or platform administration. It is

the complete daily workflow:

Receptionist registers -> token is issued -> doctor calls -> consultation starts -> consultation completes -> queue updates.

If this experience is excellent, you have the foundation of a genuinely useful product.

One final product principle: don't measure success by how many features QueuePilot has. Measure it by how little effort a clinic needs to run its day.